

**Conclusão do Diagnóstico:** O ambiente local foi submetido a uma auditoria integral e não-destrutiva cobrindo Python/uv, virtualização Docker/WSL2, Git/SSH e políticas do PowerShell. Todas as correções necessárias foram aplicadas com sucesso. O host está formalmente liberado para o **Passo 1.2**.

1. Matriz Consolidada de Prontidão do Host (Host Readiness Matrix)

| Pilar Auditado             | Configuração / Versão Real                 | Status       | Parecer e Ações Executadas                                                           |
|----------------------------|--------------------------------------------|--------------|--------------------------------------------------------------------------------------|
| Python Runtime             | Python 3.14.7 (64-bit) no PATH             | Aprovado     | CPython estável no Windows; suporte nativo a venv validado.                          |
| Gerenciador `uv`           | Astral uv/uvx v0.12.7                      | Destaque     | Capacidade de download hermético de Python e resolução ultra-rápida.                 |
| Virtualização / Containers | VT-x Ativo na BIOS / 7.15GB Livres         | Alinhado     | Docker e WSL2 ausentes no Windows; containers definidos para a VPS Hosting.          |
| Identidade Git             | <code>mteixeira23</code>                   | Corrigido    | Espaço inicial removido de <code>user.email</code> ; autoria vinculada ao GitHub.    |
| Branch Padrão Git          | <code>init.defaultBranch = main</code>     | Padronizado  | Ajustado de <code>master</code> para <code>main</code> , alinhando ao padrão GitHub. |
| Autenticação Git           | Git Credential Manager (GCM)               | Seguro       | Autenticação HTTPS com tokens OAuth protegidos pelo Windows.                         |
| Caminhos Longos Windows    | <code>LongPathsEnabled = 1</code>          | Excelente    | Sem limite de 260 caracteres no NTFS para pacotes profundos de IA.                   |
| PowerShell ExecutionPolicy | <code>CurrentUser: RemoteSigned</code>     | Desbloqueado | Permite ativação de virtualenvs ( <code>activate.ps1</code> ) sem exigir modo Admin. |
| Encoding do Python         | <code>PYTHONUTF8 = 1</code> no escopo User | Ativado      | Modo UTF-8 nativo (PEP 540) ativo no Windows, eliminando erros de acentuação.        |
| Conectividade Remota       | OpenSSH_for_Windows_9.5p2                  | Pronto       | Cliente SSH nativo disponível para acesso à VPS Hosting sem emuladores.              |

2. Decisões de Arquitetura Registradas (ADRs do Passo 1.1)

- ADR 005 — Adoção de Python 3.12 Hermético via `uv`:**

Embora o host possua o Python 3.14.7, o projeto adotará formalmente o Python 3.12 isolado via `uv venv --python 3.12` no Passo 1.2. Isso assegura 100% de paridade com o container Docker de produção ( `python:3.12-slim` ) e garante disponibilidade de wheels pré-compilados para Django, Celery e LangGraph.
- ADR 006 — Arquitetura Cloud-Native / VPS-First:**

Aprovada a estratégia de manter o computador local como estação de trabalho ágil (código, testes rápidos e Git) e concentrar a malha de containers (Docker Swarm, Traefik, PostgreSQL, Redis e RabbitMQ) diretamente na VPS Hosting Ubuntu Linux nos Passos 5 a 8, economizando RAM local e eliminando atritos de virtualização.
- ADR 007 — Proteção de Scripts Linux via `.gitattributes`:**

Diante do `core.autocrlf = true` no Windows, o projeto adotará um arquivo `.gitattributes` forçando que arquivos de script ( `*.sh` ) mantenham quebras de linha LF, impedindo falhas de execução no Docker.

### 3. Checklist de Execução Concluído (Passo 1.1)

#### Micro-Fase 1.1.1: Diagnóstico do Ecosistema Python & uv

100% CONCLUÍDO

- ✓ **Runtimes no PATH:** Python 3.14.7 identificado; venv nativo testado e aprovado.
- ✓ **Astral uv:** uv/uvx v0.12.7 ativo e pronto para gerenciar versões herméticas de Python.
- ✓ **Variáveis de Ambiente:** Confirmado que `PYTHONPATH` e `PYTHONHOME` estão limpos.

#### Micro-Fase 1.1.2: Diagnóstico de Virtualização, Docker & WSL2

100% CONCLUÍDO

- ✓ **Hardware & BIOS:** Virtualização VT-x habilitada e 7.15GB de RAM livre verificados.
- ✓ **Estratégia Definida:** Aprovada abordagem Cloud-Native com containers rodando na VPS Hostinger.

#### Micro-Fase 1.1.3: Diagnóstico e Ajustes de Git & Identidade

100% CONCLUÍDO

- ✓ **Autoria Global:** Espaço em branco corrigido em `marcos.vinicius2323@gmail.com`.
- ✓ **Branch Padrão:** `init.defaultBranch` ajustado globalmente para `main`.
- ✓ **Autenticação:** Git Credential Manager (GCM) validado para conexões HTTPS com o GitHub.

#### Micro-Fase 1.1.4: Diagnóstico e Ajustes do Shell & Permissões

100% CONCLUÍDO

- ✓ **LongPathsEnabled:** Limite de 260 caracteres do NTFS verificado como ativo (valor 1).
- ✓ **ExecutionPolicy:** Escopo `CurrentUser` ajustado para `RemoteSigned` sem privilégios de admin.
- ✓ **Permissões de I/O:** Leitura, gravação e exclusão validadas no diretório do projeto.

#### Micro-Fase 1.1.5: Consolidação & Liberação

100% CONCLUÍDO

- ✓ **Histórico Registrado:** Memória viva atualizada na `MEMORIA.md` com todas as decisões.
- ✓ **Tasklist Mestre:** Passo 1.1 assinalado como 100% concluído no `implementation_plan.md`.

### 4. Próximo Passo Imediato no Roteiro

#### 🚀 Transição para o Passo 1.2:

Com o host perfeitamente calibrado, o próximo passo consiste em instanciar o ambiente virtual hermético do projeto via `uv venv --python 3.12 .venv` e fixar a versão através do arquivo `.python-version`.